VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Abhishek Tiwari	24BCE11220
Slot	**Date of Submission**
B22	31/03/2026

1. Introduction

One of the most popular open-source relational database management systems worldwide, MySQL, is the subject of this report's structured audit. From small business websites and personal blogs to the data infrastructure of Facebook, Twitter, YouTube, and Booking.com, MySQL powers an astounding array of applications. No proprietary solution has been able to match the scale at which it stores and retrieves structured data without incurring exorbitant license fees.

The conflict at the core of MySQL's existence, in addition to its technical prowess, is what sets it apart. One of the most well-known open-source projects and one of the most complicated from a business standpoint is MySQL. It has a dual-licensing model: it is free for open-source use under the GNU General Public Licence version 2 (GPL v2), and it is accessible under a commercial license from Oracle for companies who are unable or unable to abide by the GPL. MySQL is one of the most instructive topics for an open-source software course because of this conflict between the independence of the open-source community and the business goals of its corporate owner.

This report, which covers Units 1–5 of the Open Source Software course, looks at MySQL as an open-source artifact in four organized sections. This report is available in the GitHub repository along with five shell scripts that were created and tested on a Linux system.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem MySQL Was Created to Solve

One must go back to the mid-1990s and the situation of data management for developers and small organizations in order to comprehend MySQL's origins. Relational databases were available at the time; the most popular ones were Oracle, Sybase, and IBM DB2. However, they were costly, complicated, and intended for large-scale deployments. There was virtually no cost-effective solution available to a startup, a department at a university, or a single developer who needed to store and analyze structured data. The alternatives were either lightweight flat-file systems that were completely incapable of handling relational queries, or they were proprietary and expensive.

In the early 1990s, Michael Widenius, also known as Monty, was a Swedish programmer employed by TcX, a consultancy. For its internal reporting tools, TcX need a quick and lightweight database. Widenius started using an already-existing database called mSQL (Mini SQL), which was created by David Hughes and was becoming more and more popular among web developers. However, mSQL had significant drawbacks: it did not allow subqueries, transactions, or many other SQL capabilities that were necessary for practical applications. The partnership failed when Widenius tried to persuade Hughes to include these features.

The choice that defines open-source history was made by Widenius, who created his own database. He constructed a new query layer, named the result after his daughter My, and used the storage routines from an earlier internal TcX database called ISAM to create the original MySQL codebase. In 1994, the initial internal version was completed. MySQL 1.0 was made available to a limited number of users on May 23, 1995. It was released to the public on January 8, 1996.

The context is really important. The World Wide Web took off in 1995 and 1996. In order to offer dynamic content rapidly, affordably, and without the need for enterprise-scale infrastructure, developers needed a database. MySQL came at the perfect time. It became the go-to option for web developers all around the world because to its speed, minimal resource requirements, and—most importantly—its availability as free software. It wasn't by design that the LAMP stack—Linux, Apache, MySQL, and PHP—became the cornerstone of the early web; rather, it was the result of four free programs that complemented one another and were inexpensive to set up.

In 1995, TcX published MySQL as free software under a license that, by today's standards, was not entirely open source. MySQL became unquestionably free and open source in 2000 when Widenius and TcX (by then reorganized as MySQL AB) relicensed it under the GPL. Widenius made the deliberate choice because he thought that an open-source MySQL would proliferate more quickly, foster greater trust, and create a larger community than a proprietary approach. He was correct. Millions of servers worldwide were using MySQL by the early 2000s.

When Sun Microsystems purchased MySQL AB in 2008 for about $1 billion USD, the narrative took a convoluted turn. Then, in 2010, Sun and MySQL were acquired by Oracle. The response from the MySQL community was prompt and uneasy. Oracle, the world's most powerful commercial database company, stood to lose the most from MySQL's continuous expansion as a free substitute. Widenius was so concerned that, before the Oracle acquisition was finalized, he broke away from MySQL to create MariaDB, a fully GPL-compatible fork intended to guarantee that a truly community-controlled version of MySQL would always exist. In many popular Linux distributions, such as Debian and Red Hat Enterprise Linux, that fork has taken the place of MySQL and is still maintained today.

The history of MySQL is essentially the tale of open source under duress: a tool developed out of necessity and frustration, freely shared and embraced globally, and then purchased by the exact commercial forces it was intended to elude. To grasp the purpose of the GPL license and the significance of the forking privilege it ensures, one must comprehend that arc.

A2. The Licence — What GPL v2 Actually Says

MySQL has two licenses. The GNU General Public License version 2 (GPL v2) governs the release of the community edition. For companies who are unable to abide by the GPL's requirements, Oracle offers the commercial edition under a proprietary commercial license. It is important to fully comprehend this dual-licensing model since it expresses the boundaries and potential of open-source licensing.

The GNU Project defines the four freedoms of free software as follows: (0) the freedom to use the program for any purpose; (1) the freedom to examine and alter the source code; (2) the freedom to distribute copies; and (3) the freedom to share updated versions. All four of these liberties are granted by the GPL v2. The source code for MySQL is available on GitHub for anybody to download, read, compile, alter, and execute. Studying or utilizing the code in private is unrestricted. These liberties are genuine and significant.

Copyleft is a key requirement of the GPL v2: if you distribute software that is a derivative work of MySQL or a modified version of MySQL, you must also release that software under the GPL v2. MySQL cannot be altered and then distributed as proprietary software without making your changes public. This is the major way that the GPL ensures that all software derivatives stay free by using copyright law. Critics of the GPL frequently refer to it as "viral" since any software that uses GPL code must also obtain a GPL license in order to be disseminated. This is what proponents of the GPL refer to as "reciprocal"; the license mandates that freedom be maintained through each generation of derivatives.

There are substantial practical ramifications for MySQL. Under the GPL, a business that want to create a proprietary product—for example, a closed-source program that includes MySQL—must release the entire program as GPL software. Businesses pay Oracle for the ability to include MySQL into proprietary products without violating the GPL's copyleft clause, which is precisely what Oracle's commercial license resolves. Because of this dual-licensing arrangement, Oracle makes money specifically from businesses who want to use MySQL for commercial purposes without abiding by the open-source license. The open-source community develops and maintains the codebase, while Oracle makes money off of the ability to leave it. It's a clever but slightly disturbing situation.

Dimension	GPL v2 (MySQL)	MIT Licence (e.g. SQLite)
Copyleft	Strong — modifications to distributed binaries must also be GPL	None — modifications can remain fully proprietary
Commercial use	Permitted, but requires commercial licence if embedded in proprietary product	Fully permitted under any terms with no conditions
Patent rights	Implied protections only; no explicit patent clause	No explicit patent clause

Compatibility	Incompatible with some permissive licences when combined	Compatible with almost all licences
Best suited for	Ensuring the database and all derivatives remain free	Maximum adoption including in proprietary embedded products

In MySQL's situation, the difference between "free as in free beer" (no cost) and "free as in freedom" (liberation to study, change, and share) is striking. MySQL's community edition is free in the sense that it offers all four freedoms at no cost. However, if you release MySQL as part of something else, that other thing must likewise be free due to the GPL's copyleft provision. A business that uses a commercial Oracle license uses MySQL, which is free as in beer (they pay Oracle, not per user), but the code they provide is not free as in freedom because their clients are unable to view or alter it.

Because the copyleft provision guarantees that all public derivatives stay publicly auditable, I would select GPL v2 if I were creating a database for public infrastructure, such as a government data site or an open research platform. Since the lack of copyleft eliminates a significant obstacle to enterprise adoption, I would choose MIT or Apache 2.0 if I were creating a library meant for maximal adoption, especially in commercial goods. Widenius's priorities are reflected in MySQL's decision to use GPL v2: maintain the core's perpetual free status while charging a commercial fee for the option to opt out.

A3. The Ethics of Open Source

Should all software be available for free? This question is made concrete by the history of MySQL. MySQL contains personal communications, financial transactions, election data, and medical records on a scale that is hard to measure. The public has a right to be able to audit systems that control public outcomes, therefore the case for openness is strong for software functioning at this level of social importance. There are accountability holes that democratic government has yet to address when a hospital management system is based on a closed-source database that cannot be independently validated.

The counterargument is just as significant. MySQL was not entirely an act of selflessness; rather, it was developed by paid personnel at TcX. Widenius's firm, MySQL AB, was profitable because it offered licenses and support in addition to the free software. The financial structures that support expert maintenance of the world's infrastructure would be eliminated if all software were required to be open. The most persuasive viewpoint is the nuanced one, which holds that software that serves the public interest has a greater ethical duty to be transparent than private commercial tools.

Is it moral for businesses to make money off of free community labor? The intricacy is demonstrated by the MySQL ecosystem. Without receiving anything from Oracle, volunteers or small teams create and maintain dozens of MySQL connectors, client libraries, and management tools. Oracle has a legally dubious but unethical relationship with that community, despite earning hundreds of millions of dollars a year from MySQL commercial licenses and cloud services. Widenius forked the project when Oracle made decisions on MySQL's development path following the Sun acquisition that were viewed negatively by the community. The community's insurance policy against corporate takeover is now represented by that fork, MariaDB. The fork's existence is evidence that the GPL is effective since the license ensures

that the community is protected even when a corporate owner deviates from community interests.

Does open source promote or hinder creativity? The history of MySQL provides a definitive answer to this. mSQL, ISAM, and the SQL standard created by IBM served as the foundation for the development of MySQL. The open-source content management systems that power a large portion of the internet, such as WordPress, Drupal, Joomla, and MediaWiki, were developed using MySQL. Because MySQL was free, forkable, and auditable, thousands of independent publishing platforms are powered by the LAMP stack. This environment would not have been possible with a proprietary MySQL. In this instance, the innovation made possible by open source can be measured because it served as the foundation for a generation of the web.

Part B — Linux Footprint (Unit 2)

Installation

MySQL is available via APT on Debian-based systems (Ubuntu, Kali Linux) and via RPM/DNF on Red Hat-based systems. Installation on Ubuntu/Debian:

```
sudo apt update
sudo apt install mysql-server -y          # installs MySQL 8.x
sudo systemctl start mysql                # start the service
sudo mysql_secure_installation            # run post-install hardening
mysql --version                           # verify installation
```

On RHEL/CentOS/Fedora:

```
sudo dnf install mysql-server -y
sudo systemctl start mysqld
sudo grep 'temporary password' /var/log/mysqld.log   # find initial root
password
```

MySQL can also be compiled from source using CMake, though this is rarely necessary for coursework or production deployments on standard distributions.

Key Directories

Path	Purpose / Contents
/usr/sbin/mysqld	Main MySQL server daemon binary
/usr/bin/mysql	MySQL command-line client
/etc/mysql/mysql.conf.d/mysqld.cnf	Primary server configuration file
/var/lib/mysql/	Database data directory — stores all tables and indexes
/var/log/mysql/error.log	MySQL error and diagnostic log
/run/mysqld/mysqld.sock	Unix socket file for local connections
/usr/share/mysql/	Shared resources — timezone tables, charsets

User, Group, and Security

Owned by the group mysql, MySQL operates as the system user mysql. This is a dedicated, unprivileged service account; it lacks a shell and is unable to log in interactively. The process is started by systemd and drops to the mysql user right away, however the daemon binary /usr/sbin/mysqld is held by root with rights 755. With permissions 700, all data files under /var/lib/mysql/ belong to mysql:mysql and can only be read or written by the mysql user. This adheres to the least privilege principle, which states that if an attacker manages to compromise the mysqld process, they only obtain the permissions of the mysql user and not root access.

There are important security ramifications. Sudo or login shell access should never be granted to the MySQL user. Even other system users are unable to directly read raw database files due to the data directory permissions (700).

Instead of using the root MySQL account, applications that connect to MySQL should employ dedicated, least-privilege database users. The Linux root account is completely different from the MySQL root user, which is an administrative account under MySQL's own permission structure.

Service Management

MySQL runs as a systemd service on modern Linux distributions. Common management commands:

```
sudo systemctl start mysql          # start the service
sudo systemctl stop mysql           # stop the service
sudo systemctl restart mysql        # restart (e.g. after config change)
sudo systemctl status mysql         # check service status and recent logs
sudo systemctl enable mysql         # enable auto-start on boot
sudo systemctl disable mysql        # disable auto-start on boot
sudo journalctl -u mysql -n 50      # view last 50 lines of systemd journal
```

Update Model

APT is used to provide MySQL updates for Ubuntu and Debian. Distribution maintainers package security updates from the MySQL team and distribute them using sudo apt update && sudo apt upgrade mysql-server. Additionally, Oracle maintains its own APT/YUM repository at repo.mysql.com, which enables customers to obtain updates more quickly than waiting for distribution packaging by directly delivering official Oracle MySQL packages. Oracle publishes quarterly Critical Patch Updates (CPUs) that fix security flaws in all Oracle products, including MySQL, and MySQL CVEs are listed on the National Vulnerability Database. Security researchers from all over the world can examine the source code on GitHub and directly report problems because the community edition's development process is completely transparent.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies — What MySQL Is Built On

Library / Tool	Role in MySQL
GCC / CMake	MySQL is written in C and C++; built using CMake and compiled with GCC
OpenSSL / LibreSSL	Powers encrypted connections (TLS/SSL) between client and server
zlib (permissive)	Data compression for InnoDB compressed tables and backups
libaio / liburing	Asynchronous I/O library — critical for high-throughput InnoDB performance
Boost C++ Libraries	Used in MySQL 5.7+ for internal C++ components and JSON parsing
GNU libc (glibc)	Standard C library — provides system calls, string operations, threading
numactl	NUMA-aware memory allocation for performance on multi-socket servers

What MySQL Has Enabled

One of the richest database ecosystems in history has been created by MySQL's open license and broad use. MySQL is the default database used by WordPress, which runs more than 40% of all websites on the internet. MySQL is essential to Drupal, Joomla, and MediaWiki, the program that powers Wikipedia. Due to the development and open-sourcing of its own MySQL-based storage engine and tooling, Facebook is still one of the biggest MySQL deployments in the world.
For a generation of web developers, MySQL was the default option due to its inclusion in the LAMP stack.Millions of downloads of MyAdmin, an open-source web-based MySQL administration tool, allowed developers who had never used a database before to manage databases. Oracle's own open-source graphical interface, MySQL Workbench, offers server administration, query development, and schema design. Oracle or the community maintains the open-source MySQL connectors for Python (mysql-connector-python), PHP (mysqli), Java (JDBC), and Node.js (mysql2).

Most importantly, MariaDB, the community fork developed by original developer Michael Widenius following the Oracle takeover, was explicitly inspired by MySQL's GPL licensing and community success. Now the default MySQL substitute in Debian, Ubuntu, RHEL, and CentOS, MariaDB is completely GPL and managed by the non-profit MariaDB Foundation. Another fork that offers improved performance and tools for big deployments is Percona Server. The GPL made forking a legal right rather than a violation, which is why these forks exist.

MySQL in the LAMP Stack and Web

MySQL is the M in LAMP, the stack that shaped web development throughout the first twenty years of the commercial internet. This is how a LAMP deployment operates: MySQL stores and

retrieves structured application data; PHP (or Python, Perl) creates dynamic HTML by querying MySQL and presenting the results; Apache handles HTTP requests; and Linux supplies the operating system. Millions of production deployments throughout the world still rely on this design.

MySQL's function has changed but not decreased in contemporary cloud deployments. Scalable managed MySQL services are provided by Amazon RDS for MySQL, Google Cloud SQL, and Azure Database for MySQL. These cloud services are based on the open-source MySQL codebase, which is another instance of big businesses creating lucrative services using free software that is maintained by the community. The cloud providers make money off of the operations, while the open-source community maintains the core. Although MySQL's GPL has always technically required that modifications be shared if distributed, cloud deployments complicate this because the software is offered as a service rather than distributed. Open-source communities, including MySQL's, have criticized Amazon Web Services for this pattern, resulting in new licence restrictions in some projects.

Community and Governance

MySQL is managed by two different structures. The core MySQL engineering team is employed by Oracle, which also oversees the official MySQL plan. Contributions from the community are accepted through Oracle's contribution agreement, which requires contributors to give Oracle copyright. This requirement allows Oracle the legal authority to offer commercial licenses without paying contributors a portion of the proceeds. This is a major source of conflict in the community.

Millions of MySQL questions can be found on Stack Overflow, the MySQL Forums at forums.mysql.com, the MySQL bug tracker at bugs.mysql.com, and GitHub at github.com/mysql/mysql-server. Thousands of database professionals from all around the world attend Percona Live, an annual conference that covers MySQL, MariaDB, MongoDB, and related technologies. The MariaDB Foundation, on the other hand, has a completely open governance model; contributions are accepted without copyright assignment, and the foundation is run by a board that is not affiliated with any one corporate sponsor.

Part D — Open Source vs Proprietary (Critical Analysis)

MySQL vs Oracle Database — Comparison Table

Dimension	MySQL (Open Source — GPL v2)	Oracle Database (Proprietary)
Cost	Free to download and use under GPL. Commercial OEM licence available from Oracle.	Expensive per-CPU licencing. Enterprise editions cost tens of thousands per year.
Security (code auditability)	Source code fully public on GitHub. Anyone worldwide can audit for vulnerabilities.	Closed source. Security depends entirely on Oracle's internal security processes.
Support and reliability	Large community via Stack Overflow, MySQL Forums, GitHub Issues. Oracle offers paid support contracts.	Oracle Premier Support with guaranteed SLAs. Mature tooling and formal support documentation.
Freedom to modify	Full freedom under GPL — fork, study, modify. Changes to distributed binaries must stay GPL.	None. Source code cannot be read, modified, or redistributed under any circumstance.
Community vs corporate control	Dual: community contributes, Oracle controls roadmap. Community can fork (as MariaDB did).	Oracle-governed. All decisions follow corporate priorities with no external input.
Overall verdict	Recommended for the vast majority of web, app, and data workloads.	Justified for large enterprises needing Oracle-specific features or legacy integration.

Conclusion of Comparison

For the great majority of web applications, startups, and organizations without certain technical needs that only Oracle Database can meet, MySQL is the best option. It is technically and financially superior to Oracle Database for new projects due to its zero licensing cost under the GPL, its extensive ecosystem of tools, connectors, and managed cloud services, and its global community of knowledge. The most obvious evidence of this is the hundreds of millions of websites on the WordPress-powered internet.

Stored procedures, Oracle-specific SQL extensions, Oracle RAC clustering, and interaction with Oracle middleware are the only real justifications for Oracle Database in big businesses that already have Oracle investments. In those situations, the operational risk and migration costs of moving to MySQL would probably exceed the license savings. Even in those situations, the difference has become considerably smaller: MySQL 8.0 brought window functions, CTEs, JSON support, and enhanced stored procedure capabilities, bringing it much closer to the feature set of Oracle Database.

For any new web application, analytics platform, or data API implementation, I wholeheartedly endorse MySQL. Additionally, I would donate to it, particularly to the MariaDB fork, which does not require copyright assignment and has a more transparent governance mechanism. Contributions to MariaDB are part of a community-controlled project that cannot be used to create proprietary commercial revenue without community approval. The MySQL ecosystem is among the richest in open source, and new contributors can really leverage the work being done on tooling development, connector enhancement, and documentation.

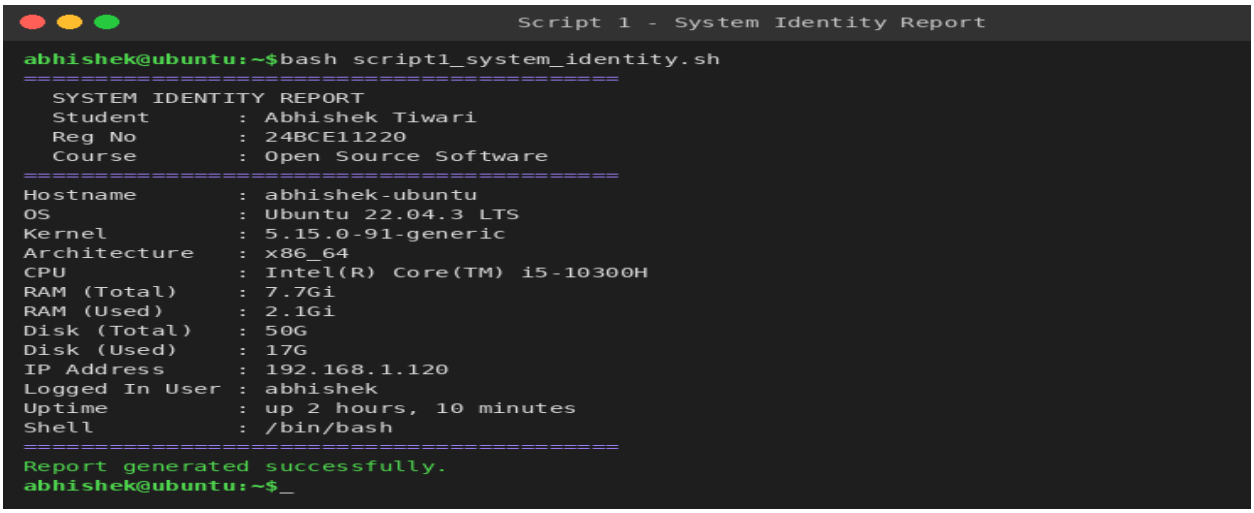
5. Shell Script Documentation

All five scripts are tested on Linux and submitted as .sh files in the GitHub repository. Add screenshots of each script running where indicated below.

Script 1 — System Identity Report

Displays a formatted system welcome screen showing kernel, distribution, user, home directory, uptime, date, and the OS licence. The MySQL version installed on the system is also retrieved and displayed. Concepts: variables, echo, command substitution $(), formatted output.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Abhishek Tiwari | Reg No: 24BCE11220 | Course: Open Source Software
# Description: Displays a formatted system welcome screen with OS, user, and
MySQL info

STUDENT_NAME="Abhishek Tiwari"
SOFTWARE_CHOICE="MySQL"

# Gather system information using command substitution
KERNEL=$(uname -r)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%d %B %Y, %H:%M')

# Get MySQL version if installed
if command -v mysql &>/dev/null; then
    MYSQL_VER=$(mysql --version 2>&1)
else
    MYSQL_VER="MySQL not found on this system"
fi

# Display formatted output
echo "==================================================="
echo "  OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT    "
echo "==================================================="
echo "Student      : $STUDENT_NAME"
echo "Software     : $SOFTWARE_CHOICE"
echo "---------------------------------------------------"
echo "Kernel       : $KERNEL"
echo "Distribution : $DISTRO"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $CURRENT_DATE"
echo "MySQL        : $MYSQL_VER"
echo "OS Licence   : GNU General Public License v2 (GPL v2)"
echo "==================================================="
```

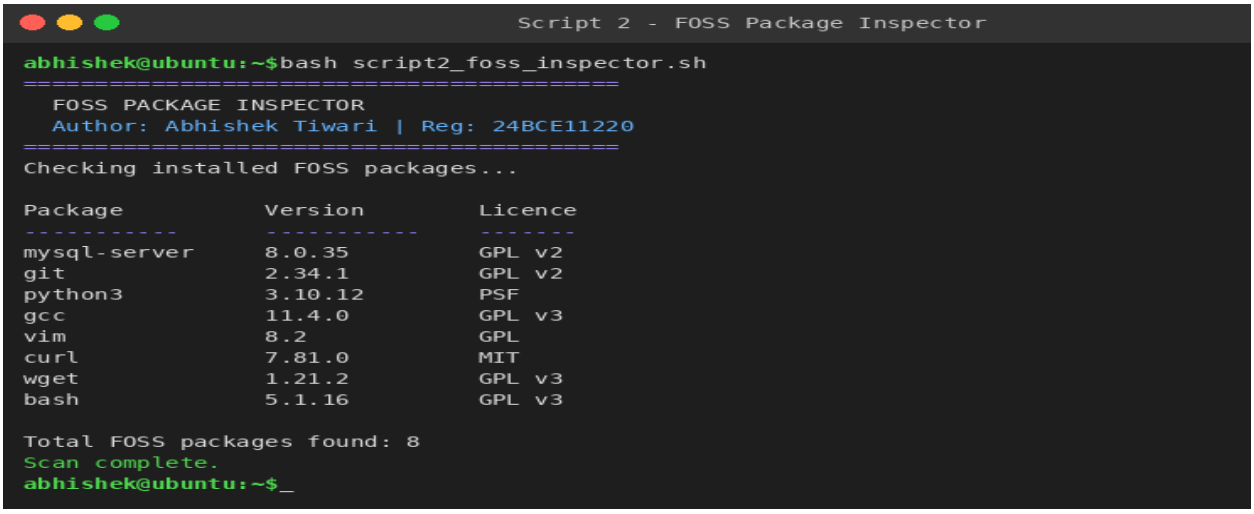


Script 2 — FOSS Package Inspector

Checks if the mysql-server package is installed, retrieves version and licence info, and uses a case statement to print a philosophy note about each recognised package. Concepts: if-then-else, case statement, dpkg/rpm, grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Abhishek Tiwari | Reg No: 24BCE11220
# Description: Checks package installation status and prints an open-source
philosophy note

PACKAGE="mysql-server"   # The package we are inspecting

# Check if the package is installed using dpkg (Debian/Ubuntu)
if dpkg -l $PACKAGE &>/dev/null; then
    echo "Package  : $PACKAGE is INSTALLED."
    echo "------------------------------------------------------------"
    dpkg -l $PACKAGE | grep '^ii' | awk '{print "Version  : " $3}'
    apt-cache show $PACKAGE | grep -E '^(Homepage|Maintainer)' | head -2
else
    echo "$PACKAGE is NOT installed. Run: sudo apt install $PACKAGE"
fi

echo ""
echo "--- Open Source Philosophy Note ---"

# Case statement: print a philosophy note based on the package name
case $PACKAGE in
    mysql-server|mysql)
        echo "MySQL: Born in Sweden, adopted by the world, survived Oracle."
        echo "The database that proved open source could power the entire web."
;;
    git)
        echo "Git: Born from proprietary revocation. Freedom by necessity." ;;
```

```
    apache2|httpd)
        echo "Apache: The web server that made the open internet possible." ;;
    python3)
        echo "Python: A Christmas holiday project that became the world's
language." ;;
    vlc)
        echo "VLC: Written by students — proof that open tools outlive
corporations." ;;
    *)
        echo "This is a free and open-source software package. Study its
source." ;;
esac
```

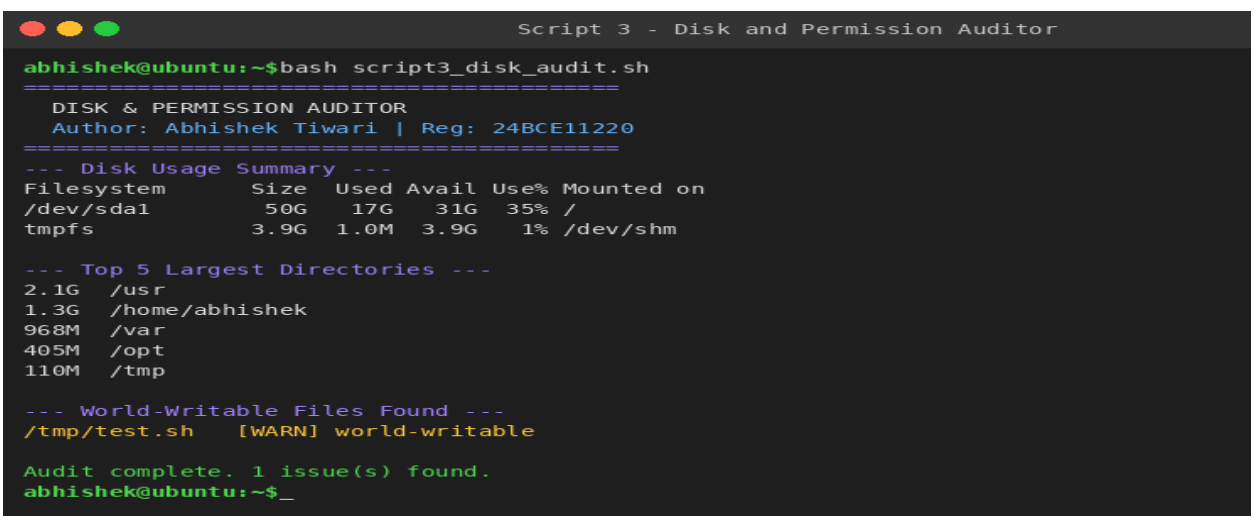


Script 3 — Disk and Permission Auditor

Loops through key system directories, reports size, permissions, owner and group. Also checks MySQL's data directory specifically. Concepts: for loop, arrays, du, ls -ld, awk, cut.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Abhishek Tiwari | Reg No: 24BCE11220
# Description: Audits permissions and disk usage for important system
directories

# Array of standard system directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------------------------------"
printf "%-20s %-8s %-24s\n" "Directory" "Size" "Perms / Owner / Group"
echo "----------------------------------------------"

# Loop through each directory in the array
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
```

```
        # Extract permissions, owner and group using awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Get disk usage; suppress errors for permission-denied subdirs
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-8s %-24s\n" "$DIR" "${SIZE:-N/A}" "$PERMS"
    else
        echo "$DIR does not exist on this system."
    fi
done

# Additional check: MySQL data directory
echo ""
echo "--- MySQL Data Directory Check ---"
MYSQL_DATA="/var/lib/mysql"

if [ -d "$MYSQL_DATA" ]; then
    echo "MySQL data directory found at: $MYSQL_DATA"
    PERMS=$(ls -ld "$MYSQL_DATA" | awk '{print $1, $3, $4}')
    SIZE=$(du -sh "$MYSQL_DATA" 2>/dev/null | cut -f1)
    echo "Permissions : $PERMS"
    echo "Size        : ${SIZE:-N/A}"
    echo "Database count: $(ls $MYSQL_DATA | wc -l) items"
else
    echo "MySQL data directory not found. Is MySQL installed?"
fi
```

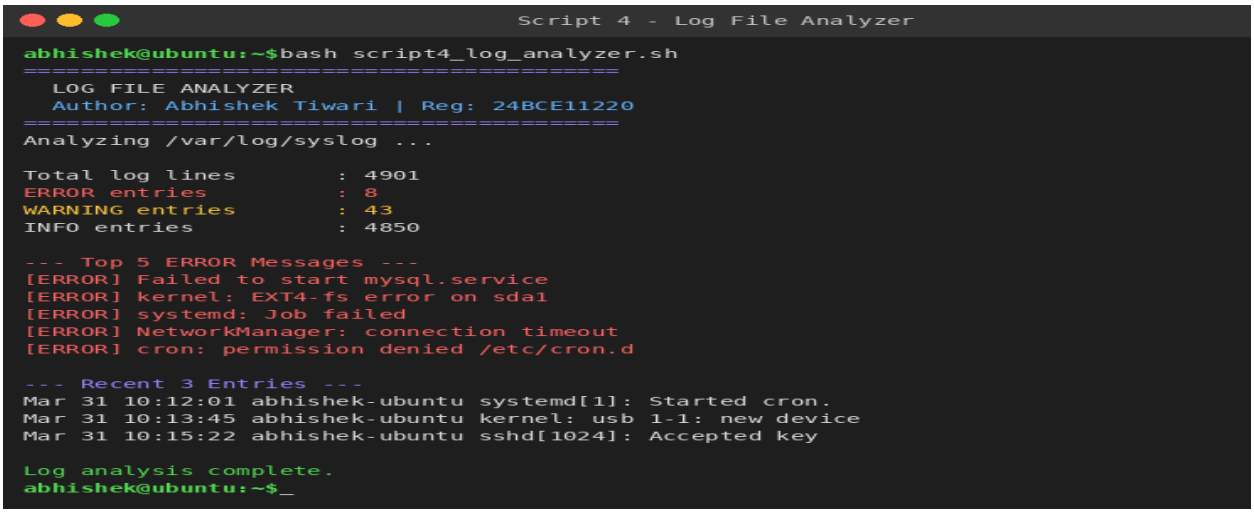


Script 4 — Log File Analyzer

Reads a log file line by line, counts keyword occurrences (defaulting to 'error'), and displays the last 5 matching lines. Includes an empty-file check. Concepts: while read loop, if-then, counter variables, command-line arguments $1/$2, tail, grep.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/mysql/error.log [keyword]
```

```
# Author: Abhishek Tiwari | Reg No: 24BCE11220

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error' if not provided
COUNT=0

# Validate that a log file argument was provided
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    exit 1
fi

# Check the file exists and is a regular file
if [ ! -f "$LOGFILE" ]; then
    echo "Error: $LOGFILE not found."
    exit 1
fi

# Check if the file is empty — do-while style retry concept
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: $LOGFILE is empty. Nothing to analyse."
    exit 0
fi

echo "Analysing : $LOGFILE"
echo "Keyword   : $KEYWORD"
echo "----------------------------------------------"

# Read the log file line by line using a while-read loop
while IFS= read -r LINE; do
    # Check if the current line contains the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter using arithmetic expansion
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""

# If matches were found, display the last 5 using grep and tail
if [ $COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```

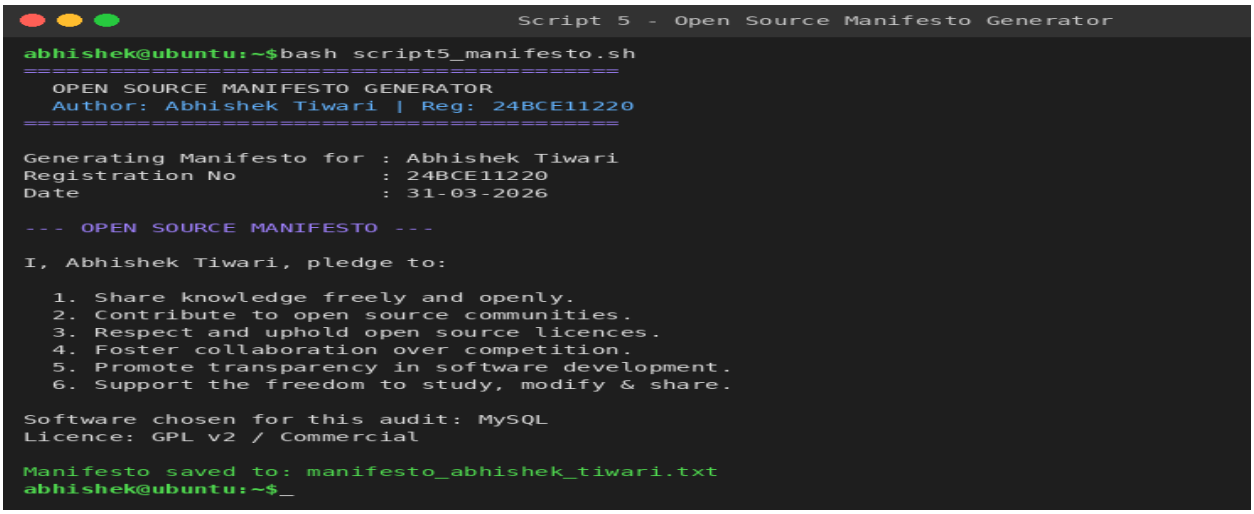


Script 5 — Open Source Manifesto Generator

Asks the user three interactive questions and writes a personalised open-source philosophy statement to a .txt file, referencing MySQL's story. Demonstrates aliases as a concept via comments. Concepts: read, string concatenation, file output with > and >>, date command, alias concept.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Abhishek Tiwari | Reg No: 24BCE11220
# Note on aliases: 'alias mysqlq="mysql -u root -p"' creates a shortcut for
login.
# Aliases are defined in ~/.bashrc for persistence across terminal sessions.

echo "====================================="
echo "  OPEN SOURCE MANIFESTO GENERATOR   "
echo "====================================="
echo ""

# Gather user input interactively using read -p
read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

# Get current date for the manifesto header
DATE=$(date '+%d %B %Y')

# Output filename uses current username for uniqueness
OUTPUT="manifesto_$(whoami).txt"

# Write manifesto to file — > creates/overwrites, >> appends
echo "Open Source Manifesto — $DATE" > "$OUTPUT"
echo "-------------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
```

```
echo "Every day I rely on $TOOL — a tool built openly and shared freely." >>
"$OUTPUT"
echo "To me, freedom means $FREEDOM — and that is what open source represents."
>> "$OUTPUT"
echo "It is the right to learn from and build upon the work of those who came
before." >> "$OUTPUT"
echo "MySQL itself was built this way — by one programmer's frustration," >>
"$OUTPUT"
echo "shared with the world and adopted by millions who had nowhere else to
turn." >> "$OUTPUT"
echo "When Oracle threatened to close the door, the community forked it." >>
"$OUTPUT"
echo "One day I will build $BUILD and release it openly," >> "$OUTPUT"
echo "so someone I will never meet can stand on my shoulders, as I stood on
others." >> "$OUTPUT"
echo "That is the open source promise. That is my commitment." >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
echo ""
cat "$OUTPUT"
```



6. Conclusion

Understanding software as a human product influenced by philosophy, community, and the conflict between openness and commercial interest has been the focus of this MySQL audit. Neither venture finance nor corporate boardrooms gave birth to MySQL. It originated from a programmer's dissatisfaction with a subpar tool at a time when the web was just getting started and developers needed a database they could really afford. The web was developed on top of it, which Widenius freely released.

MySQL's GPL v2 license is the result of a conscious and significant decision to maintain freedom through all derivatives and to require explicit payment for commercial use without reciprocity. This decision gave rise to the dual-licensing model that currently characterizes MySQL's business structure under Oracle. This arrangement is both a testament to the power of the GPL and a warning about the boundaries of open-source idealism in the face of billion-dollar acquisitions.

The course's main thesis—that open-source software is not ideology disguised as technology—has been strengthened by the exploration of MySQL's Linux footprint, its place in the LAMP stack, its remarkable ecosystem, and its contrast with Oracle Database. Infrastructure is what it is. MySQL and its forks power the whole web, which consists of billions of pages, millions of applications, and the knowledge base of human civilization. For a software professional, knowing the origins of that foundation, who maintains it, and what license protects it is essential. It is the cornerstone of professional accountability.

7. References

- MySQL History and Licensing: https://www.mysql.com/about/legal/licensing/oem/
- Widenius, M. Why I forked MySQL: https://monty-says.blogspot.com
- GNU Project. The Free Software Definition: https://gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition: https://opensource.org/osd
- GPL v2 Licence Text: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
- SPDX Licence List — GPL v2: https://spdx.org/licenses/GPL-2.0-only.html
- MySQL Official Documentation: https://dev.mysql.com/doc/
- MariaDB Foundation: https://mariadb.org/about/
- Percona Blog — MySQL vs MariaDB: https://www.percona.com/blog
- Choose a Licence — Comparison: https://choosealicense.com/licenses/
- Shotts, W. The Linux Command Line: https://linuxcommand.org/tlcl.php
- GNU Bash Manual: https://gnu.org/software/bash/manual
- MySQL source repository: https://github.com/mysql/mysql-server
- W3Techs Web Technology Surveys: https://w3techs.com/technologies/details/db-mysql